

Lab 14 — EF Core: CRUD, Relationships, and Loading

Name: SaifElden Khaled Nazmy Lotfy

Assignment: Lab 14 | ITI Summer Training — Web Development Using .NET

Lab ID: 31

Fingerprinted values for Lab ID 31

Part	Formula	Result
C — GPA value	$3.0 + ((31 \bmod 7) \times 0.1) = 3.0 + (3 \times 0.1)$	3.3
E — Delete behaviour	$31 \bmod 2 = 1$	SetNull (FK made nullable)
F — Extra courses	$(31 \bmod 3) + 2 = 1 + 2$	3

Part A — Setup

Steps: Ran Session14_Prelnit.sql (read-only) and confirmed it reports the database found, Students with rows, and AssignedCourseName still present. Pressed run before touching any TODO and confirmed the four Session 13 students printed correctly, confirming the connection string.

```
info: 7/30/2026 05:08:23.421 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (34ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [s].[Id], [s].[FullName], [s].[Gpa], [s].[YearOfStudy]
      FROM [Students] AS [s]
      Yara Adel - Year 2, GPA 3.50
      Omar Hesham - Year 3, GPA 2.80
      Nada Samir - Year 1, GPA 3.90
      Kareem Fouad - Year 4, GPA 3.20
```

- Students reported by Prelnit script: Yara Adel, Omar Hesham, Nada Samir, Kareem Fouad
- Migrations already applied before today: InitialCreate, AddCreditsCompleted

Part B — Predict-the-Output Drills

B1. Code

```
var s = await context.Students.FirstAsync(x => x.Id == 1);
s.Gpa = 3.99;
Console.WriteLine(s.Gpa);
```

Prediction: Prints 3.99 in the console, but the database row is unchanged.

Reason: Assigning to s.Gpa only changes the in-memory C# object. EF's change tracker notes the entity now differs from its loaded snapshot, but no SQL is generated or sent until SaveChangesAsync() runs — which never happens in this snippet.

B2. Code

```
var instructors = await context.Instructors.ToListAsync();
foreach (var i in instructors)
    Console.WriteLine($"{i.FullName}: {i.Courses.Count}");
```

Prediction: Every line prints ": 0", even for instructors who really do teach courses.

Reason: Instructors.ToListAsync() with no Include() only fetches Instructor rows. Courses was initialized to new(), so reading it is safe (no NullReferenceException) — it's just still empty, because nothing ever asked EF to populate it.

B3. Code

```
var s = await context.Students.AsNoTracking().FirstAsync();  
s.Gpa = 2.0;  
await context.SaveChangesAsync();
```

Prediction: Nothing happens to the database — no error, no change, no exception.

Reason: `AsNoTracking()` tells EF not to snapshot this entity. With no snapshot, `SaveChangesAsync()` has nothing to diff s against, detects zero changed entities, and sends no SQL. This is dangerous precisely because it's silent.

Part C — Full CRUD, Async, Verified (Fingerprinted — GPA = 3.3)

Formula: $3.0 + ((31 \bmod 7) \times 0.1) = 3.0 + (3 \times 0.1) = 3.3$

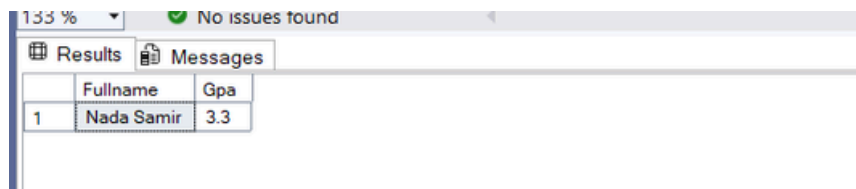
C1–C6. Steps

- **Read** — Nada Samir read via FirstAsync; GPA printed.
- **Change (unsaved)** — `nada.Gpa = 3.3` printed from C#, SSMS still shows the old value. The assignment only changed the in-memory object; EF sends no SQL until `SaveChangesAsync()`.
- **Save** — after `SaveChangesAsync()`, SSMS now reads 3.3. EF diffs the snapshot vs. current values, so the generated UPDATE touches only the Gpa column.
- **Create** — new student, my own name, Year 2, GPA 3.3. Id was 0 before save (CLR default); after `SaveChangesAsync()`, EF read back the database-assigned identity value.
- **Update** — YearOfStudy changed to 3, saved, verified in SSMS.
- **Delete** — own student removed and saved, verified gone. `Remove()` has no async version because it only flags the entity Deleted in-memory; the actual DELETE is sent inside `SaveChangesAsync()`, which is already the async call.



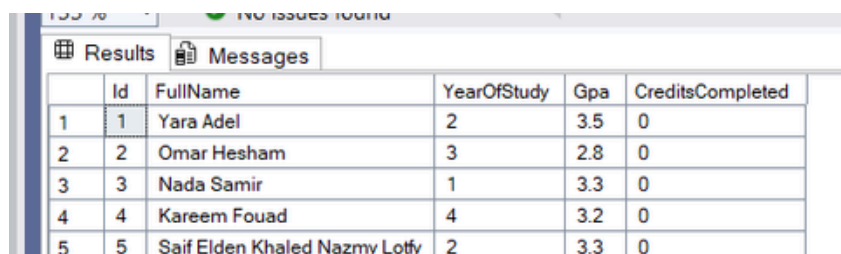
133 % No issues found

	FullName	Gpa
1	Nada Samir	3.9



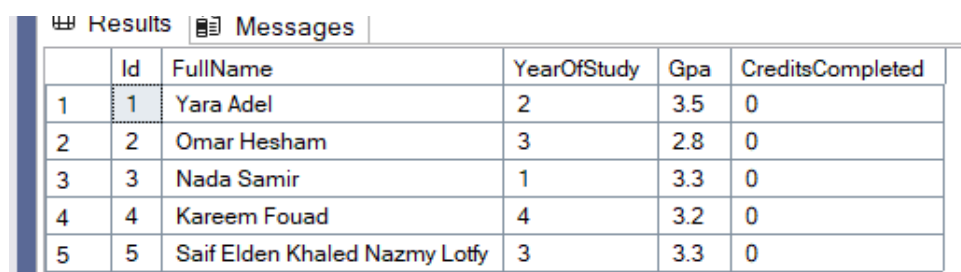
133 % No issues found

	FullName	Gpa
1	Nada Samir	3.3

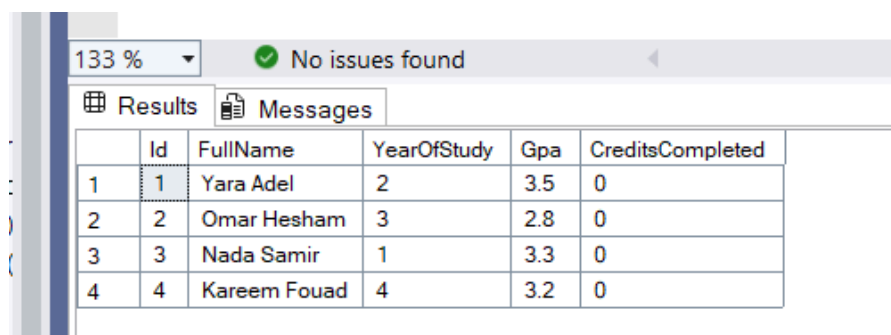


133 % No issues found

	Id	FullName	YearOfStudy	Gpa	CreditsCompleted
1	1	Yara Adel	2	3.5	0
2	2	Omar Hesham	3	2.8	0
3	3	Nada Samir	1	3.3	0
4	4	Kareem Fouad	4	3.2	0
5	5	Saif Elden Khaled Nazmy Lotfy	2	3.3	0



	Id	FullName	YearOfStudy	Gpa	CreditsCompleted
1	1	Yara Adel	2	3.5	0
2	2	Omar Hesham	3	2.8	0
3	3	Nada Samir	1	3.3	0
4	4	Kareem Fouad	4	3.2	0
5	5	Saif Elden Khaled Nazmy Lotfy	3	3.3	0



133 % No issues found

	Id	FullName	YearOfStudy	Gpa	CreditsCompleted
1	1	Yara Adel	2	3.5	0
2	2	Omar Hesham	3	2.8	0
3	3	Nada Samir	1	3.3	0
4	4	Kareem Fouad	4	3.2	0

Part D — Constraints, Both Ways, Migrated onto Real Data

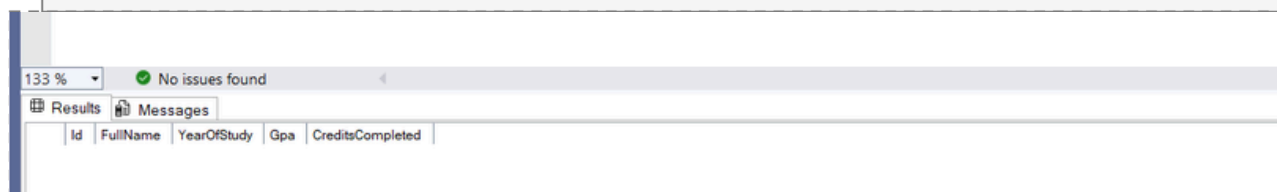
- [Required] + [MaxLength(100)] added to Student.FullName.
- [Required] + [MaxLength(150)] added to Course.CourseName.
- The same FullName rule repeated via the Fluent API in OnModelCreating.

Migration read before applying (AddStudentConstraints)

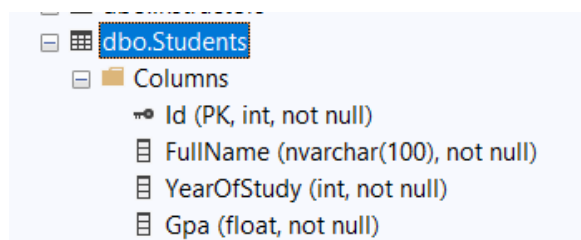
- **Operation on FullName:** AlterColumn<string>(…) — narrows nvarchar(max) down to nvarchar(100)
- **nullable / oldNullable:** nullable: false, oldNullable: false
- **Two kinds of existing row that could break it:** (1) FullName IS NULL, (2) LEN(FullName) > 100.

SSMS check query run first:

```
SELECT * FROM Students WHERE FullName IS NULL OR LEN(FullName) > 100;
```



Applied with Update-Database; column type/nullability confirmed changed in SSMS. **Proof:** wrapped a null FullName insert in try/catch — the database refused it.



```
6         at Microsoft.EntityFrameworkCore.SqlServer.Storage.Internal.SqlServerExecutionStrategy.ExecuteAsync<T>(
7         ult](TState state, Func`4 operation, Func`4 verifySucceeded, CancellationTok
8         at Microsoft.EntityFrameworkCore.DbContext.SaveChangesAsync(Boolean acceptAllChangesOnSuccess, Cance
9         n cancellationToken)
10        [D] Database refused a NULL FullName. Exception type: DbUpdateException
11
12        C:\Users\PandaEyes\Desktop\C#\Lab14\bin\Debug\net10.0\StudentPortalConsole.exe (process 14300) exited with c
13        Press any key to close this window . . .
14        // Read-before-apply questions for the
```

Part E — The Real Relationship (Fingerprinted — $31 \bmod 2 = 1 \rightarrow \text{SetNull}$)

Lab ID mod 2	Configured OnDelete	Why
1	DeleteBehavior.SetNull	Deleting an instructor should clear the link, not destroy or block courses. Requires a null

- Added InstructorId (int?) and an Instructor? navigation property to Course.
- Added a Courses list to Instructor, initialized to new(). Deleted AssignedCourseName entirely — keeping both would leave two competing sources of truth
- Configured the relationship with HasOne/WithMany/HasForeignKey/OnDelete(SetNull).

Migration read before applying (AddInstructorCourseRelationship)

• **Every operation Up() performs:** DropColumn("AssignedCourseName", "Instructors");
AddColumn<int>("InstructorId", "Courses", nullable: true); AddForeignKey(...); CreateIndex on Courses.InstructorId.

• **The data-destroying step:** DropColumn("AssignedCourseName", "Instructors") — every instructor's existing AssignedCourseName text is permanently lost.

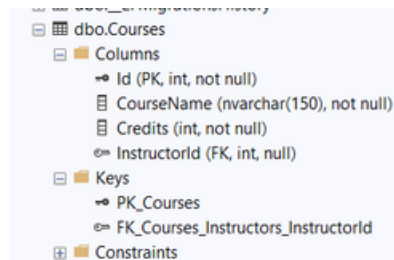
• **What a real project would do first:** capture that data before dropping it — back it up, or better, use the existing values to populate InstructorId on matching courses so the information migrates instead of vanishing.

Applied. InstructorId column confirmed in SSMS; FK constraint found under the table's Keys node.

FK constraint name: FK_Courses_Instructors_InstructorId

Linked "Web Development Using .NET" to Hamdy by setting webCourse.InstructorId = hamdy.Id directly — the navigation property was never loaded or assigned.

Proved the constraint is real: tried InstructorId = 9999 inside a try/catch. The database rejected it. **Compare to AssignedCourseName:** the old string column would have silently accepted "9999" — it was just text, checked against nothing.



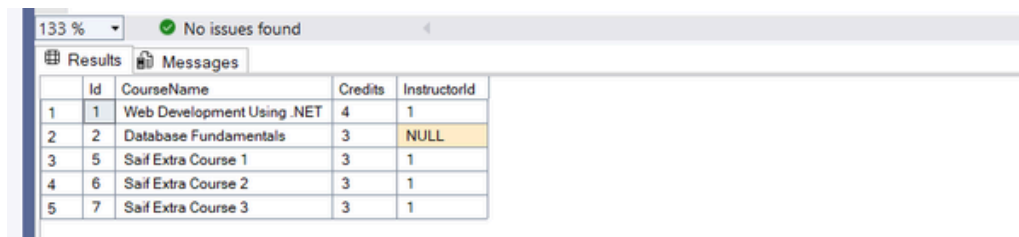
	Id	CourseName	Credits	InstructorId
1	1	Web Development Using .NET	4	1
2	2	Database Fundamentals	3	NULL

```
at Microsoft.EntityFrameworkCore.ChangeTracking.Internal.StateManager.SaveChangesAsync(StateManager stateManager, Boolean acceptAllChangesOnSuccess, CancellationToken cancellationToken)
at Microsoft.EntityFrameworkCore.SqlServer.Storage.Internal.SqlServerExecutionStrategy.ExecuteAsync[TState,TResult](TState state, Func`4 operation, Func`4 verifySucceeded, CancellationToken cancellationToken)
at Microsoft.EntityFrameworkCore.DbContext.SaveChangesAsync(Boolean acceptAllChangesOnSuccess, CancellationToken cancellationToken)
[E] Invalid InstructorId (9999) rejected: The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Courses_Instructors_InstructorId". The conflict occurred in database "ITI_StudentPortal_EF", table "dbo.Instructors", column 'Id'.
```

Part F — Loading Strategies and N+1 (Fingerprinted — 3 extra courses)

Formula: $(31 \bmod 3) + 2 = 1 + 2 = 3$

- Created 3 extra courses, all assigned to Hamdy.
- SQL logging enabled via `LogTo(...)` in `OnConfiguring`.
- **Without Include:** every instructor printed 0 courses. Query count: 1 (a plain `SELECT` against `Instructors`). The data exists; nothing asked EF to fetch it.
- **With Include:** correct counts for every instructor. Query count: 1 (a single `SELECT` with a `LEFT JOIN`).
- **Why SQL returned more rows than instructors:** a `JOIN` returns one row per matching pair. EF's relational fixup folds the duplicated rows back into one `Instructor` object with a populated `Courses` list.
- **Explicit loading:** one instructor loaded (`Courses.Count = 0`), then `Entry(...).Collection(...).LoadAsync()` — count now correct. Query count: 2 total, worse than `Include`'s single query here, but the right tool when related data is only sometimes needed.
- **AsNoTracking():** loaded all students read-only, changed one, called `SaveChangesAsync()`. Result: row unchanged in SSMS, no error at all — exactly what B3 predicted.



	Id	CourseName	Credits	InstructorId
1	1	Web Development Using .NET	4	1
2	2	Database Fundamentals	3	NULL
3	5	Saif Extra Course 1	3	1
4	6	Saif Extra Course 2	3	1
5	7	Saif Extra Course 3	3	1

```
Fullinfo: 7/30/2026 07:18:51.062 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
Executed DbCommand (39ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
SELECT [i].[Id], [i].[FullName], [i].[YearsOfExperience]
FROM [Instructors] AS [i]
Hamdy: 0 courses
Mona Khalil: 0 courses
C:\Users\PandaEyes\Desktop\C#\Lab14\bin\Debug\net10.0\StudentPortalConsole.exe (process 20864) exited with code 0 (0x0).
Press any key to close this window . . .
```

```
Fullinfo: 7/30/2026 07:20:37.079 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
Executed DbCommand (33ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
SELECT [i].[Id], [i].[FullName], [i].[YearsOfExperience], [c].[Id], [c].[CourseName]
FROM [Instructors] AS [i]
LEFT JOIN [Courses] AS [c] ON [i].[Id] = [c].[InstructorId]
ORDER BY [i].[Id]
Hamdy:
Web Development Using .NET (4 credits)
Saif Extra Course 1 (3 credits)
Saif Extra Course 2 (3 credits)
Saif Extra Course 3 (3 credits)
Mona Khalil:
```

```
Fullinfo: 7/30/2026 07:25:35.705 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
Executed DbCommand (36ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
SELECT TOP(1) [i].[Id], [i].[FullName], [i].[YearsOfExperience]
FROM [Instructors] AS [i]
[F] Hamdy: 0 courses (before explicit load)
Fullinfo: 7/30/2026 07:25:35.908 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
Executed DbCommand (17ms) [Parameters=[@p='1' (Nullable = true)], CommandType='Text', CommandTimeout='30']
SELECT [c].[Id], [c].[CourseName], [c].[Credits], [c].[InstructorId]
FROM [Courses] AS [c]
WHERE [c].[InstructorId] = @p
[F] Hamdy: 4 courses (after explicit load)
```

Part G — Wrap-Up Reflection

1. Lab ID 31, three derived values, with arithmetic:

- Part C GPA: $3.0 + ((31 \bmod 7) \times 0.1) = 3.0 + (3 \times 0.1) = 3.3$
- Part E delete rule: $31 \bmod 2 = 1 \rightarrow \text{SetNull}$
- Part F extra count: $(31 \bmod 3) + 2 = 1 + 2 = 3$

2. My OnDelete question (SetNull): SetNull keeps the child row (the course) alive after its parent (the instructor) is deleted, by clearing the link rather than deleting the course (Cascade) or refusing the delete (Restrict). Clearing the link means writing NULL into InstructorId — and a plain int column can never legally hold NULL in SQL Server. So the FK has to be declared int? (and the navigation Instructor?) before SetNull is even a legal configuration; with a non-nullable FK, EF would reject this setup outright.

3. Did any migration fail today? no rollback but the two commands that would have been used are still Update-Database <PreviousMigrationName> and Remove-Migration.

4. Multiple enumeration (Session 13) vs. N+1 (today) — what they share: both are caused by code that looks like it touches the database once but doesn't. Multiple enumeration re-ran an unmaterialized LINQ query once per consumption (Count(), then foreach, then Average() — three round-trips from what read like one query). N+1 does the same thing shaped differently: looping over instructor.Courses with no Include(), under lazy loading, fires one extra query per instructor rather than per re-read of the same query. The fix in both cases has the same shape too: make the round-trips visible (.ToList() a query once; .Include() a navigation up front) and collapse many round-trips into one.

Evidence from my own Part F counts: TODO 16/17 both logged 1 query because lazy loading is off in this project, while TODO 18's explicit load pushed the count to 2 for a single instructor.